1. Student Group Formation with Score Balancing

Problem Statement

Arrange students' scores into groups of a given size and try to keep the group totals as close as possible.

Objectives

• Read student scores and group size.
• Form groups using a simple greedy method.
• Display each group's total and the difference between the highest and lowest totals.

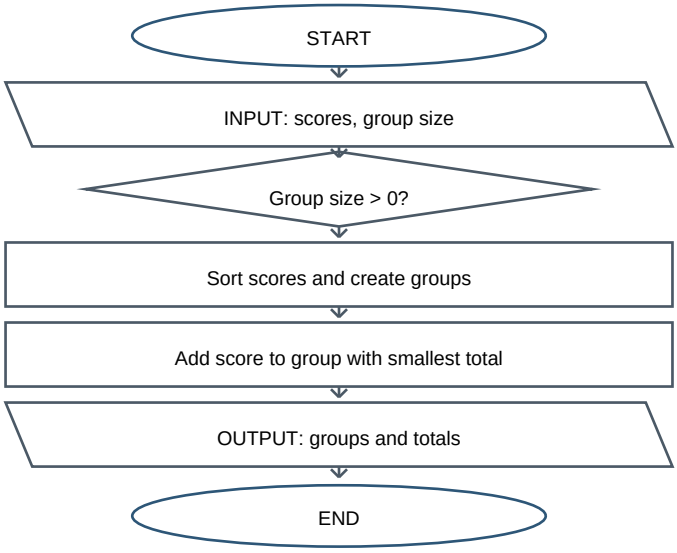
Assumptions

• Scores are non-negative integers.
• Scores are sorted from highest to lowest and each score is placed in the group with the smallest current total.
• The final group may contain fewer students if the count is not divisible by group size.

Algorithm

1. Input scores and group size.
2. Sort scores in descending order.
3. Create groups and add each score to the group with the smallest total.
4. Print groups, totals, and the difference.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
INPUT scores, group_size
IF group_size <= 0: PRINT error; STOP
SORT scores in descending order
CREATE empty groups and totals
FOR each score:
    choose group with smallest total
    add score and update total
PRINT groups, totals, max(total)-min(total)
STOP
```

Python Source Code

```
scores = [80, 70, 60, 50, 40, 30]
group_size = 2

if group_size <= 0:
    print("Group size must be positive")
else:
    scores.sort(reverse=True)
    count = (len(scores) + group_size - 1) // group_size
    groups = [[] for _ in range(count)]
    totals = [0] * count

    for score in scores:
        i = totals.index(min(totals))
        groups[i].append(score)
        totals[i] += score

    for i in range(count):
        print("Group", i + 1, ":", groups[i], "Total =", totals[i])
```

```
print("Difference =", max(totals) - min(totals))
```

Test Cases

Input / Test	Expected result
Scores [80,70,60,50,40,30], size 2	Groups printed; difference shown.
Group size 0	Error message shown.

Sample Output

```
Group 1 : [80, 30] Total = 110
Group 2 : [70, 40] Total = 110
Group 3 : [60, 50] Total = 110
Difference = 0
```

Conclusion

A simple greedy method balances group totals, although it may not always give the mathematically best grouping.

2. Smart Canteen Order and Stock Management

Problem Statement

Process customer orders, calculate bills, reduce stock, reject unavailable quantities, and show daily sales and remaining stock.

Objectives

• Store item prices and stock.
• Check each order before accepting it.
• Calculate bills and summarize sales.

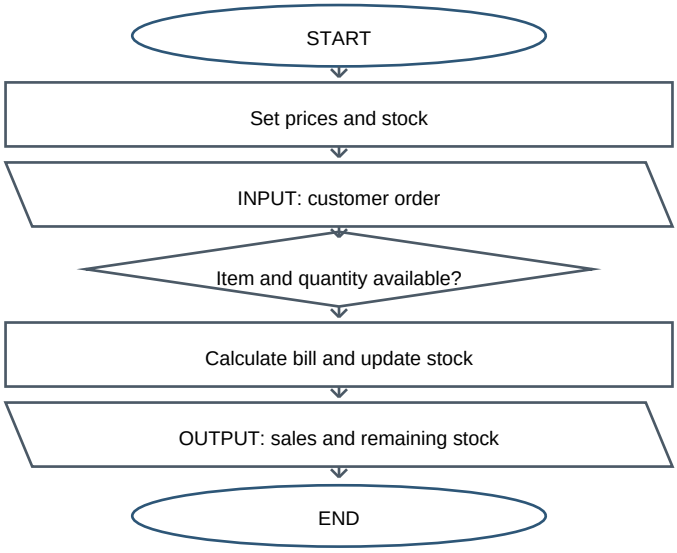
Assumptions

• Items are identified by names.
• Each order contains an item and quantity.
• An order is accepted only if the item exists, quantity is positive, and stock is sufficient.

Algorithm

1. Create dictionaries for price and stock.
2. Read each order.
3. Validate item and quantity; calculate bill and reduce stock if valid.
4. Print order results, total sales, and remaining stock.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
SET prices and stock
SET total_sales = 0
FOR each order (item, quantity):
    IF item missing OR quantity <= 0 OR quantity > stock:
        PRINT rejected
    ELSE:
        bill = price * quantity
        reduce stock and add bill to total_sales
        PRINT bill
PRINT total_sales and stock
STOP
```

Python Source Code

```
prices = {"idli": 20, "dosa": 40, "juice": 30}
stock = {"idli": 20, "dosa": 10, "juice": 8}
orders = [("idli", 2), ("dosa", 3), ("juice", 10)]
total_sales = 0

for item, quantity in orders:
    if item not in stock or quantity <= 0 or quantity > stock.get(item, 0):
        print("Rejected:", item, quantity)
    else:
        bill = prices[item] * quantity
        stock[item] -= quantity
        total_sales += bill
        print("Accepted:", item, "Bill =", bill)

print("Daily sales =", total_sales)
print("Remaining stock =", stock)
```

Test Cases

Input / Test	Expected result
('idli', 2)	Accepted; bill = 40.
('juice', 10)	Rejected because stock is 8.
('dosa', 3)	Accepted; bill = 120.

Sample Output

```
Accepted: idli Bill = 40
Accepted: dosa Bill = 120
Rejected: juice 10
Daily sales = 160
Remaining stock = {'idli': 18, 'dosa': 7, 'juice': 8}
```

Conclusion

The program prevents selling more items than available and updates sales and stock after accepted orders.

3. Library Borrowing and Fine Reconciliation

Problem Statement

Check borrowed-book records, calculate fines for late returns, list overdue books, and update availability.

Objectives

• Read book ID, allowed days, days kept, and return status.
• Calculate fines for late returns.
• Show whether each book is available or borrowed.

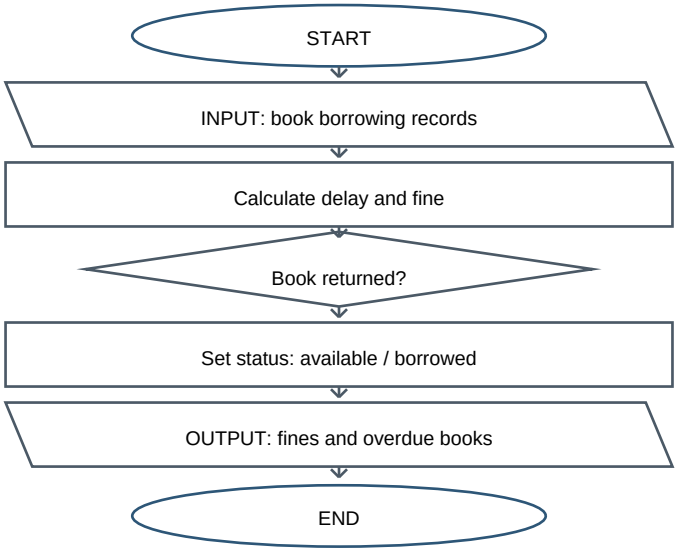
Assumptions

• Fine is ■5 per late day.
• Records are tuples: (book_id, allowed_days, days_kept, returned).
• A returned book is available.

Algorithm

1. Read each tuple record.
2. Calculate delay as max(0, days kept - allowed days).
3. Calculate fine and identify overdue books.
4. Show availability using the return status.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
SET fine_rate = 5
FOR each record:
    delay = max(0, days_kept - allowed_days)
    fine = delay * fine_rate
    IF delay > 0: mark overdue
    IF returned: status = AVAILABLE
    ELSE: status = BORROWED
    PRINT record and fine
PRINT overdue books
STOP
```

Python Source Code

```
records = [
    ("B101", 7, 9, True),
    ("B102", 5, 4, True),
    ("B103", 10, 12, False)
]
fine_rate = 5
overdue_books = []

for book_id, allowed, kept, returned in records:
    delay = max(0, kept - allowed)
    fine = delay * fine_rate
    if delay > 0:
        overdue_books.append(book_id)
    status = "Available" if returned else "Borrowed"
    print(book_id, "Fine =", fine, "Status =", status)

print("Overdue books =", overdue_books)
```

Test Cases

Input / Test	Expected result
B101: allowed 7, kept 9, returned	Fine ■10; available.
B102: allowed 5, kept 4	No fine.
B103: allowed 10, kept 12, not returned	Fine ■10; borrowed.

Sample Output

```
B101 Fine = 10 Status = Available
B102 Fine = 0 Status = Available
B103 Fine = 10 Status = Borrowed
Overdue books = ['B101', 'B103']
```

Conclusion

The program calculates late fines and separates available books from books that are still borrowed.

4. Campus Transport Allocation

Problem Statement

Calculate the buses needed for each route from passenger demand and bus capacity, and find unserved passengers if only a fixed number of buses are available.

Objectives

• Calculate required buses.
• Calculate available seats.
• Find unserved passengers.

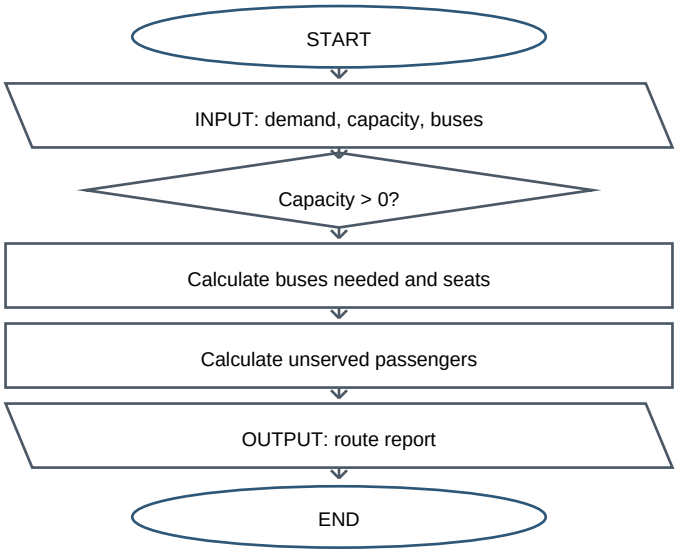
Assumptions

• All buses on a route have the same capacity.
• A bus is a whole vehicle.
• The number of available buses is supplied.

Algorithm

1. Read demand, capacity, and available buses.
2. Calculate buses needed using ceiling division.
3. Calculate available seats.
4. Find unserved passengers and print results.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
FOR each route:
    INPUT demand, capacity, buses_available
    IF capacity <= 0: PRINT error
    ELSE:
        needed = ceiling(demand / capacity)
        seats = buses_available * capacity
        unserved = max(0, demand - seats)
        PRINT needed and unserved
STOP
```

Python Source Code

```
import math

routes = [
    ("Route A", 95, 40, 2),
    ("Route B", 60, 30, 2),
    ("Route C", 25, 20, 1)
]

for name, demand, capacity, buses_available in routes:
    if capacity <= 0:
        print(name, "Invalid capacity")
    else:
        needed = math.ceil(demand / capacity)
        seats = buses_available * capacity
        unserved = max(0, demand - seats)
        print(name, "Buses needed =", needed,
              "Unserved =", unserved)
```

Test Cases

Input / Test	Expected result
Demand 95, capacity 40, 2 buses	3 needed; 15 unserved.
Demand 60, capacity 30, 2 buses	2 needed; none unserved.
Capacity 0	Invalid capacity.

Sample Output

```
Route A Buses needed = 3 Unserved = 15
Route B Buses needed = 2 Unserved = 0
Route C Buses needed = 2 Unserved = 5
```

Conclusion

The program estimates bus requirements and shows where extra transport capacity is needed.

5. Manufacturing Quality Control Matrix

Problem Statement

Analyse production and defective-unit counts across batches, calculate defect percentages, and flag batches above a threshold.

Objectives

• Store quantities in matrices.
• Calculate defect percentage per batch.
• Identify batches above the threshold.

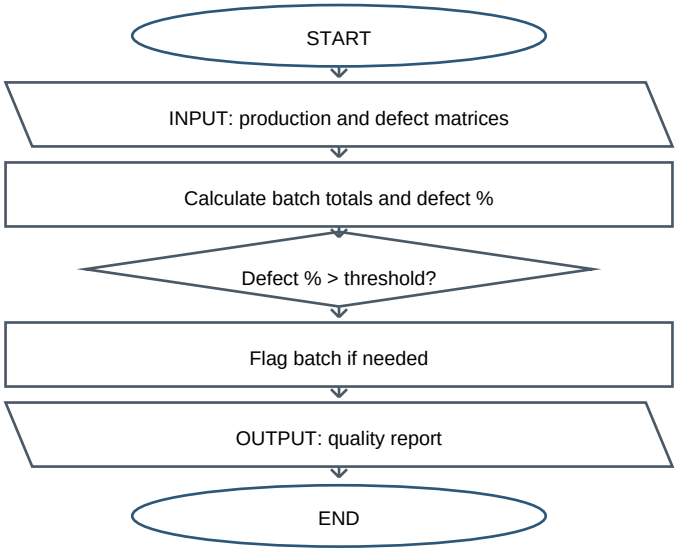
Assumptions

• Each row is one batch.
• Production and defect matrices have the same size.
• A batch is flagged if its defect percentage is greater than the threshold.

Algorithm

1. Store production and defect matrices.
2. Add production quantities and defects for each batch.
3. Calculate percentage of defects.
4. Flag batches above the threshold.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
FOR each batch:
    total = sum(production row)
    bad = sum(defect row)
    IF total > 0: percent = bad / total * 100
    ELSE: percent = 0
    IF percent > threshold: flag batch
    PRINT batch and percent
STOP
```

Python Source Code

```
production = [
    [100, 150, 200],
    [80, 100, 120],
    [200, 200, 100]
]
defects = [
    [2, 3, 5],
    [8, 5, 7],
    [1, 2, 2]
]
threshold = 5

for i in range(len(production)):
    total = sum(production[i])
    bad = sum(defects[i])
    percent = (bad / total) * 100 if total > 0 else 0
    status = "CHECK" if percent > threshold else "OK"
    print("Batch", i + 1, "Defect % =", round(percent, 2), status)
```

Test Cases

Input / Test	Expected result
Batch 1: 10 defects / 450 units	2.22%; OK.
Batch 2: 20 defects / 300 units	6.67%; CHECK.
Threshold 5%	Only values above 5% are flagged.

Sample Output

```
Batch 1 Defect % = 2.22 OK
Batch 2 Defect % = 6.67 CHECK
Batch 3 Defect % = 1.0 OK
```

Conclusion

The matrix method helps compare batches and identify which production batches need quality checks.

6. Nested List Examination Seating Validation

Problem Statement

Validate a seating matrix for duplicate roll numbers, reserved seats, empty seats, and invalid allocations.

Objectives

• Check duplicate roll numbers.
• Count empty and reserved seats.
• Print a simple validation report.

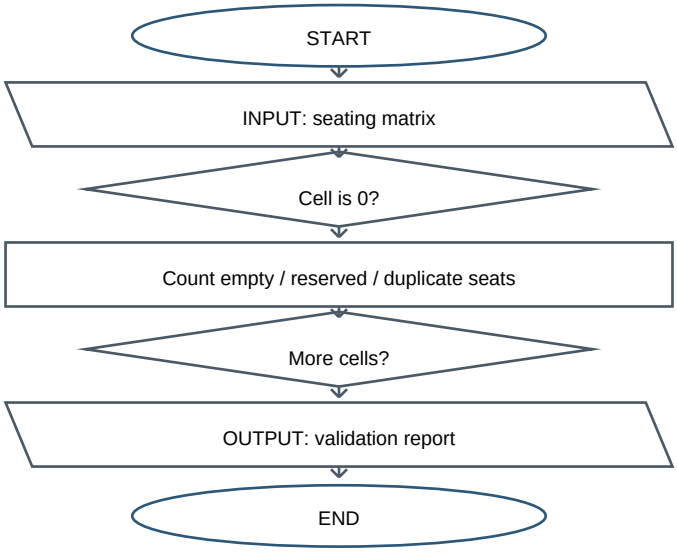
Assumptions

• 0 means empty.
• Negative values represent reserved seats.
• Positive values are roll numbers and should appear only once.

Algorithm

1. Visit every cell in the matrix.
2. Count empty and reserved seats.
3. Use a set to track roll numbers.
4. Report duplicates and counts.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
SET seen = empty set; empty = 0; reserved = 0
FOR each cell:
    IF cell == 0: empty += 1
    ELSE IF cell < 0: reserved += 1
    ELSE IF cell in seen: record duplicate
    ELSE: add cell to seen
PRINT counts and duplicates
STOP
```

Python Source Code

```
seats = [
    [101, 102, 0],
    [103, -1, 102],
    [104, 0, 105]
]
seen = set()
duplicates = []
empty = 0
reserved = 0

for row in seats:
    for roll in row:
        if roll == 0:
            empty += 1
        elif roll < 0:
            reserved += 1
        elif roll in seen:
            if roll not in duplicates:
                duplicates.append(roll)
```

```
        else:
            seen.add(roll)

print("Empty seats =", empty)
print("Reserved seats =", reserved)
print("Duplicate roll numbers =", duplicates)
```

Test Cases

Input / Test	Expected result
Roll number 102 appears twice	Duplicate 102 reported.
Cell value 0	Counted as empty.
Cell value -1	Counted as reserved.

Sample Output

```
Empty seats = 2
Reserved seats = 1
Duplicate roll numbers = [102]
```

Conclusion

Nested loops can check every seat and find common seating-allocation mistakes.

7. Water Distribution Fairness Analysis

Problem Statement

Compare water supply with demand across areas and days, calculate shortages, and find the most underserved area.

Objectives

• Calculate shortage per area.
• Compare total shortages.
• Identify the area with the largest shortage.

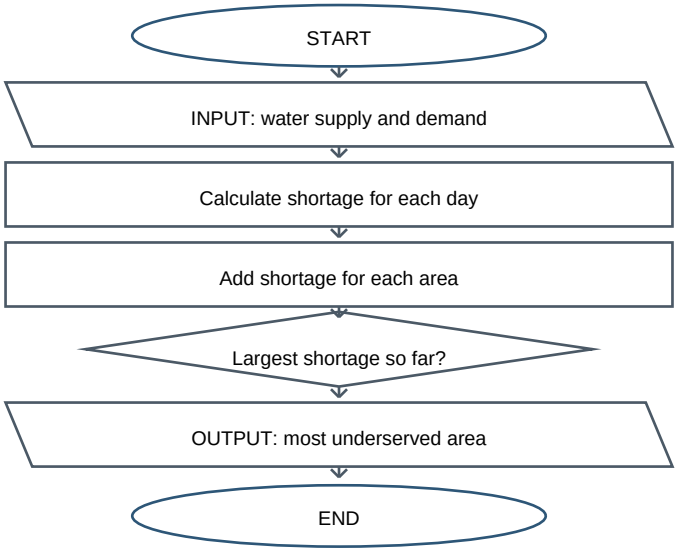
Assumptions

• Supply and demand use the same unit.
• Rows represent areas and columns represent days.
• Shortage is max(0, demand - supply).

Algorithm

1. Store supply and demand matrices.
2. Calculate shortage for each day.
3. Add shortages for each area.
4. Print the area with the largest shortage.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
FOR each area:
    total_shortage = 0
    FOR each day:
        shortage = max(0, demand - supply)
        total_shortage += shortage
    PRINT area and total_shortage
SELECT area with maximum shortage
PRINT area
STOP
```

Python Source Code

```
areas = ["North", "South", "East"]
supply = [[80, 70, 90], [60, 50, 55], [100, 90, 95]]
demand = [[100, 100, 100], [80, 80, 80], [100, 100, 100]]
totals = []

for i in range(len(areas)):
    shortage = 0
    for day in range(len(supply[i])):
        shortage += max(0, demand[i][day] - supply[i][day])
    totals.append(shortage)
    print(areas[i], "Total shortage =", shortage)

worst = totals.index(max(totals))
print("Most underserved area =", areas[worst])
```

Test Cases

Input / Test	Expected result
North supply 80,70,90; demand 100 each day	Shortage = 60.
South supply 60,50,55; demand 80 each day	Shortage = 75.
Compare area totals	South has the largest shortage.

Sample Output

```
North Total shortage = 60
South Total shortage = 75
East Total shortage = 15
Most underserved area = South
```

Conclusion

Shortage totals help identify areas that need more water supply.

8. Multi-Product Price Fluctuation Analysis

Problem Statement

Compare old and new prices, calculate percentage changes, find the largest fluctuation, and flag changes above a threshold.

Objectives

• Calculate percentage changes.
• Find the largest absolute fluctuation.
• Flag products above a threshold.

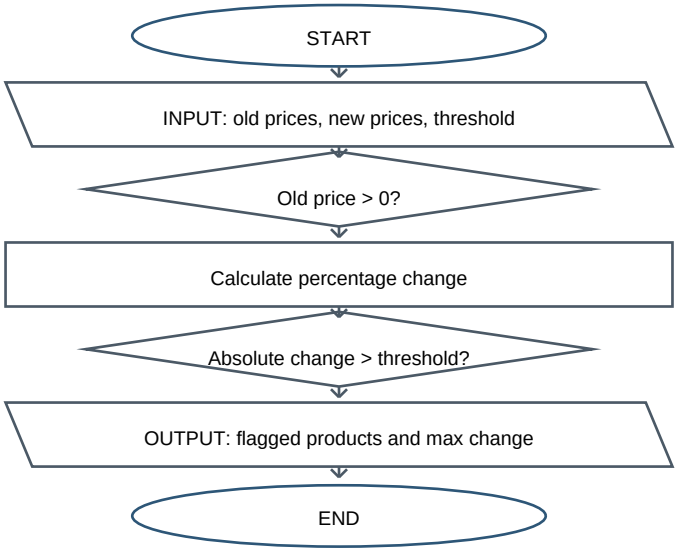
Assumptions

• Old prices must be greater than zero.
• Percentage change = (new - old) / old × 100.
• Absolute percentage change is used to detect both increases and decreases.

Algorithm

1. Read product names and old/new prices.
2. Calculate percentage change.
3. Flag if absolute change exceeds threshold.
4. Print the largest fluctuation.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
FOR each product:
    IF old_price <= 0: PRINT invalid
    ELSE:
        change = (new - old) / old * 100
        IF abs(change) > threshold: flag product
        PRINT change
PRINT largest fluctuation
STOP
```

Python Source Code

```
products = ["Pen", "Book", "Bag"]
old_prices = [10, 50, 500]
new_prices = [12, 45, 600]
threshold = 15
changes = []

for i in range(len(products)):
    old = old_prices[i]
    new = new_prices[i]
    if old <= 0:
        print(products[i], "Invalid old price")
        continue
    change = ((new - old) / old) * 100
    changes.append((products[i], change))
    flag = "FLAG" if abs(change) > threshold else "OK"
    print(products[i], "Change =", round(change, 2), "%", flag)

if changes:
    largest = max(changes, key=lambda item: abs(item[1]))
```

```
print("Largest fluctuation =", largest[0], round(largest[1], 2), "%")
```

Test Cases

Input / Test	Expected result
Pen price 10 to 12	20% increase; flagged at 15%.
Book price 50 to 45	-10%; not flagged.
Bag price 500 to 600	20% increase; tied largest fluctuation.

Sample Output

```
Pen Change = 20.0 % FLAG
Book Change = -10.0 % OK
Bag Change = 20.0 % FLAG
Largest fluctuation = Pen 20.0 %
```

Conclusion

Percentage change allows fair comparison between products with different starting prices.

9. Tuple-Based Course Registration Validation

Problem Statement

Check tuple-based course registrations, find duplicates, and identify students selecting prohibited course combinations.

Objectives

• Find repeated student-course pairs.
• Check course clashes.
• Print invalid combinations.

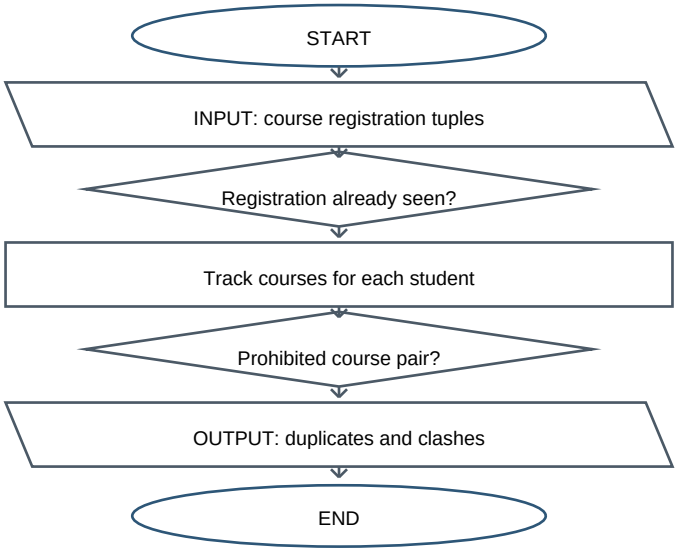
Assumptions

• Each registration is (student_id, course).
• A student should register for each course only once.
• AI and ML clash in this example.

Algorithm

1. Read registration tuples.
2. Track student-course pairs to find duplicates.
3. Group courses by student.
4. Check for the prohibited AI and ML pair.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
SET seen_pairs = empty set
FOR each (student, course):
    IF pair already seen: record duplicate
    ADD pair to seen_pairs
    ADD course to student's course set
FOR each student:
    IF AI and ML both selected: report clash
PRINT duplicates and clashes
STOP
```

Python Source Code

```
registrations = [
    ("S101", "AI"),
    ("S101", "ML"),
    ("S102", "Python"),
    ("S101", "AI"),
    ("S103", "AI")
]
seen = set()
duplicates = []
student_courses = {}

for student, course in registrations:
    pair = (student, course)
    if pair in seen and pair not in duplicates:
        duplicates.append(pair)
    seen.add(pair)
    student_courses.setdefault(student, set()).add(course)
print("Duplicate registrations =", duplicates)
```

```
for student, courses in student_courses.items():
    if "AI" in courses and "ML" in courses:
        print("Invalid combination for", student, ": AI and ML clash")
```

Test Cases

Input / Test	Expected result
S101 registers for AI twice	Duplicate (S101, AI) found.
S101 selects AI and ML	Timetable clash reported.
S102 selects Python only	No invalid combination.

Sample Output

```
Duplicate registrations = [('S101', 'AI')]
Invalid combination for S101 : AI and ML clash
```

Conclusion

Tuples, sets, and dictionaries help check registrations and detect repeated or conflicting choices.

10. Campus Resource Allocation Under Constraints

Problem Statement

Compare equipment needs with available stock, allocate equipment by priority, and report shortages.

Objectives

• Process activities in priority order.
• Avoid allocating more than available stock.
• Report shortages and remaining stock.

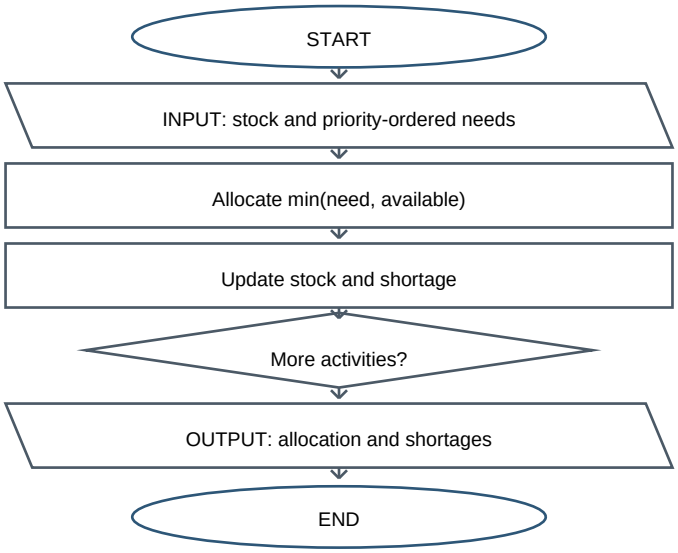
Assumptions

• Activities are already sorted from highest to lowest priority.
• Each activity needs one type of equipment.
• Higher-priority activities receive equipment first.

Algorithm

1. Set available equipment quantity.
2. Read each activity and requirement in priority order.
3. Allocate the smaller of requirement and available stock.
4. Update stock and calculate shortage.
5. Print allocation results.

Flowchart (text inside standard symbols)



Flowchart symbol key: Oval = Start/End | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision

Pseudocode

```
START
SET available = initial_stock
FOR each activity in priority order:
    allocated = min(required, available)
    shortage = required - allocated
    available = available - allocated
    PRINT activity, allocated, shortage
PRINT remaining stock
STOP
```

Python Source Code

```
available = 15
activities = [
    ("Lab exam", 8),
    ("Workshop", 6),
    ("Club event", 5)
]

for name, required in activities:
    allocated = min(required, available)
    shortage = required - allocated
    available -= allocated
    print(name, "Allocated =", allocated, "Shortage =", shortage)

print("Remaining equipment =", available)
```

Test Cases

Input / Test	Expected result
Lab exam needs 8; stock 15	Allocate 8; remaining 7.
Workshop needs 6; stock 7	Allocate 6; remaining 1.
Club event needs 5; stock 1	Allocate 1; shortage 4.

Sample Output

```
Lab exam Allocated = 8 Shortage = 0
Workshop Allocated = 6 Shortage = 0
Club event Allocated = 1 Shortage = 4
Remaining equipment = 0
```

Conclusion

Priority-based allocation gives resources to important activities first and identifies unmet requirements.